

Denoising

```
from keras.layers import Input, Dense
from keras.models import Model
from keras.datasets import fashion_mnist
from keras.callbacks import ModelCheckpoint, EarlyStopping,
ReduceLROnPlateau
import numpy as np
import matplotlib.pyplot as plt

# this is the size of our encoded representations
encoding_dim = 32 # 32 floats -> compression of factor 24.5, assuming
the input is 784 floats

# this is our input placeholder
input_img = Input(shape=(784,))
# "encoded" is the encoded representation of the input
encoded = Dense(encoding_dim, activation='relu')(input_img)
encoded = Dense(128, activation='relu')(encoded)
encoded = Dense(28, activation='relu')(encoded)
# "decoded" is the lossy reconstruction of the input
decoded = Dense(28, activation='relu')(encoded) # added
decoded = Dense(128, activation='sigmoid')(decoded)
decoded = Dense(784, activation='sigmoid')(decoded)
# this model maps an input to its reconstruction
autoencoder = Model(input_img, decoded)
# this model maps an input to its encoded representation
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

(x_train, _), (x_test, _) = fashion_mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = x_train.reshape((len(x_train), np.prod(x_train.shape[1:])))
x_test = x_test.reshape((len(x_test), np.prod(x_test.shape[1:])))

#introducing noise
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0,
scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0,
scale=1.0, size=x_test.shape)

# Callbacks
# ModelCheckpoint to save the best model during training
model_checkpoint = ModelCheckpoint('best_autoencoder_model.keras',
monitor='val_loss', save_best_only=True)
# EarlyStopping to stop training when validation loss doesn't improve
for 5 consecutive epochs
```

```
early_stopping = EarlyStopping(monitor='val_loss', patience=5,  
                                restore_best_weights=True)  
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.1,  
                                patience=3)
```

```
autoencoder.fit(x_train_noisy, x_train,  
                epochs=100,  
                batch_size=32,  
                shuffle=True,  
                validation_data=(x_test_noisy, x_test),  
                callbacks=[early_stopping, model_checkpoint])
```

Epoch 1/100

1875/1875 ————— 7s 3ms/step - loss: 0.3985 - val_loss: 0.3231

Epoch 2/100

1875/1875 ————— 4s 2ms/step - loss: 0.3177 - val_loss: 0.3138

Epoch 3/100

1875/1875 ————— 4s 2ms/step - loss: 0.3099 - val_loss: 0.3093

Epoch 4/100

1875/1875 ————— 6s 3ms/step - loss: 0.3072 - val_loss: 0.3071

Epoch 5/100

1875/1875 ————— 4s 2ms/step - loss: 0.3033 - val_loss: 0.3065

Epoch 6/100

1875/1875 ————— 5s 2ms/step - loss: 0.3016 - val_loss: 0.3036

Epoch 7/100

1875/1875 ————— 4s 2ms/step - loss: 0.3004 - val_loss: 0.3031

Epoch 8/100

1875/1875 ————— 5s 2ms/step - loss: 0.2999 - val_loss: 0.3015

Epoch 9/100

1875/1875 ————— 4s 2ms/step - loss: 0.2985 - val_loss: 0.3012

Epoch 10/100

1875/1875 ————— 4s 2ms/step - loss: 0.2974 - val_loss: 0.3005

Epoch 11/100

1875/1875 ————— 5s 2ms/step - loss: 0.2972 - val_loss: 0.3008

Epoch 12/100

1875/1875 ————— 4s 2ms/step - loss: 0.2968 - val_loss: 0.3000

Epoch 13/100

1875/1875 ————— 4s 2ms/step - loss: 0.2960 - val_loss:

```
0.2987
Epoch 14/100
1875/1875 ————— 4s 2ms/step - loss: 0.2946 - val_loss:
0.2987
Epoch 15/100
1875/1875 ————— 6s 3ms/step - loss: 0.2945 - val_loss:
0.2983
Epoch 16/100
1875/1875 ————— 5s 2ms/step - loss: 0.2955 - val_loss:
0.2981
Epoch 17/100
1875/1875 ————— 4s 2ms/step - loss: 0.2938 - val_loss:
0.2981
Epoch 18/100
1875/1875 ————— 6s 3ms/step - loss: 0.2937 - val_loss:
0.2978
Epoch 19/100
1875/1875 ————— 4s 2ms/step - loss: 0.2940 - val_loss:
0.2976
Epoch 20/100
1875/1875 ————— 4s 2ms/step - loss: 0.2930 - val_loss:
0.2976
Epoch 21/100
1875/1875 ————— 6s 2ms/step - loss: 0.2942 - val_loss:
0.2973
Epoch 22/100
1875/1875 ————— 4s 2ms/step - loss: 0.2934 - val_loss:
0.2974
Epoch 23/100
1875/1875 ————— 5s 2ms/step - loss: 0.2928 - val_loss:
0.2969
Epoch 24/100
1875/1875 ————— 4s 2ms/step - loss: 0.2932 - val_loss:
0.2966
Epoch 25/100
1875/1875 ————— 4s 2ms/step - loss: 0.2941 - val_loss:
0.2966
Epoch 26/100
1875/1875 ————— 4s 2ms/step - loss: 0.2923 - val_loss:
0.2969
Epoch 27/100
1875/1875 ————— 5s 2ms/step - loss: 0.2924 - val_loss:
0.2962
Epoch 28/100
1875/1875 ————— 4s 2ms/step - loss: 0.2923 - val_loss:
0.2966
Epoch 29/100
1875/1875 ————— 5s 2ms/step - loss: 0.2920 - val_loss:
0.2962
```

```
Epoch 30/100
1875/1875 ————— 4s 2ms/step - loss: 0.2923 - val_loss:
0.2964
Epoch 31/100
1875/1875 ————— 5s 2ms/step - loss: 0.2925 - val_loss:
0.2960
Epoch 32/100
1875/1875 ————— 6s 3ms/step - loss: 0.2921 - val_loss:
0.2960
Epoch 33/100
1875/1875 ————— 4s 2ms/step - loss: 0.2918 - val_loss:
0.2959
Epoch 34/100
1875/1875 ————— 4s 2ms/step - loss: 0.2920 - val_loss:
0.2955
Epoch 35/100
1875/1875 ————— 5s 2ms/step - loss: 0.2915 - val_loss:
0.2955
Epoch 36/100
1875/1875 ————— 4s 2ms/step - loss: 0.2922 - val_loss:
0.2956
Epoch 37/100
1875/1875 ————— 4s 2ms/step - loss: 0.2920 - val_loss:
0.2955
Epoch 38/100
1875/1875 ————— 5s 2ms/step - loss: 0.2914 - val_loss:
0.2958
Epoch 39/100
1875/1875 ————— 4s 2ms/step - loss: 0.2923 - val_loss:
0.2955
Epoch 40/100
1875/1875 ————— 5s 2ms/step - loss: 0.2919 - val_loss:
0.2956
Epoch 41/100
1875/1875 ————— 4s 2ms/step - loss: 0.2916 - val_loss:
0.2957
Epoch 42/100
1875/1875 ————— 5s 2ms/step - loss: 0.2915 - val_loss:
0.2954
Epoch 43/100
1875/1875 ————— 6s 2ms/step - loss: 0.2916 - val_loss:
0.2955
Epoch 44/100
1875/1875 ————— 4s 2ms/step - loss: 0.2916 - val_loss:
0.2959
Epoch 45/100
1875/1875 ————— 4s 2ms/step - loss: 0.2919 - val_loss:
0.2959
Epoch 46/100
```

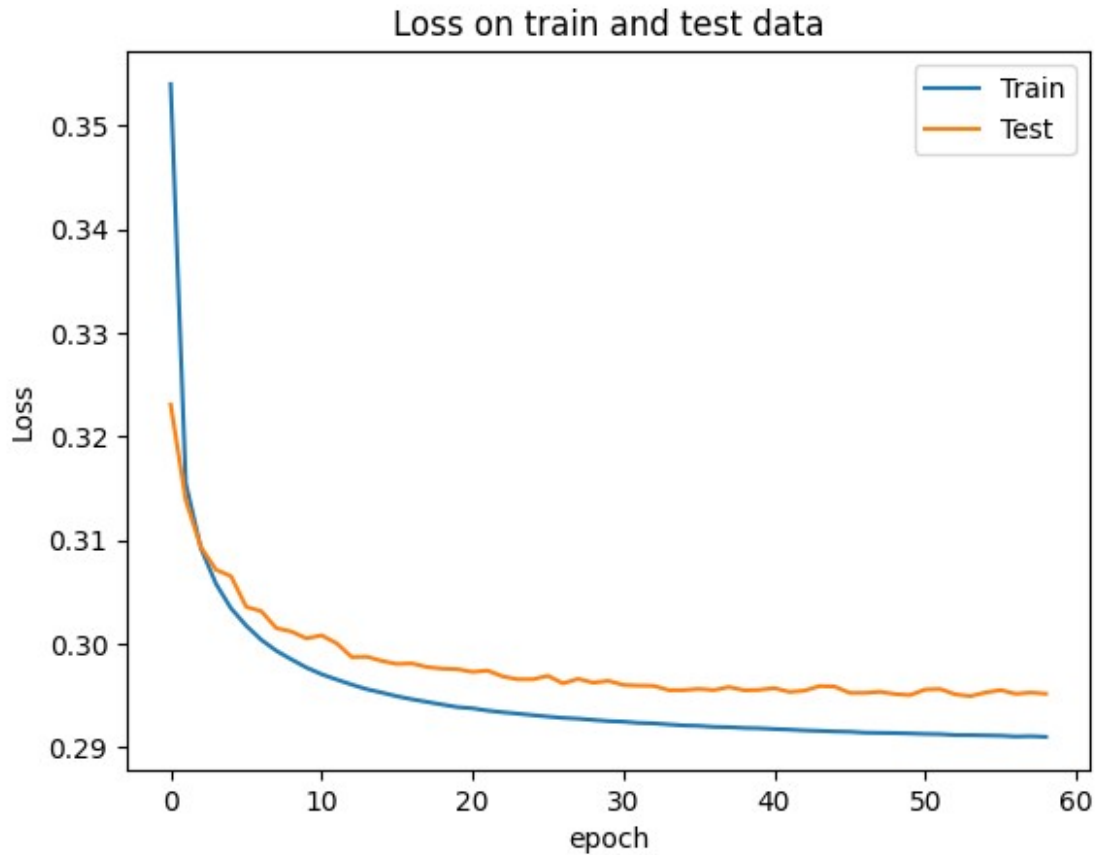
```
1875/1875 ————— 6s 2ms/step - loss: 0.2908 - val_loss: 0.2953
Epoch 47/100
1875/1875 ————— 4s 2ms/step - loss: 0.2906 - val_loss: 0.2953
Epoch 48/100
1875/1875 ————— 6s 2ms/step - loss: 0.2911 - val_loss: 0.2954
Epoch 49/100
1875/1875 ————— 4s 2ms/step - loss: 0.2908 - val_loss: 0.2951
Epoch 50/100
1875/1875 ————— 4s 2ms/step - loss: 0.2914 - val_loss: 0.2951
Epoch 51/100
1875/1875 ————— 6s 3ms/step - loss: 0.2913 - val_loss: 0.2956
Epoch 52/100
1875/1875 ————— 4s 2ms/step - loss: 0.2906 - val_loss: 0.2956
Epoch 53/100
1875/1875 ————— 4s 2ms/step - loss: 0.2914 - val_loss: 0.2951
Epoch 54/100
1875/1875 ————— 6s 2ms/step - loss: 0.2909 - val_loss: 0.2949
Epoch 55/100
1875/1875 ————— 4s 2ms/step - loss: 0.2916 - val_loss: 0.2953
Epoch 56/100
1875/1875 ————— 5s 2ms/step - loss: 0.2909 - val_loss: 0.2955
Epoch 57/100
1875/1875 ————— 5s 2ms/step - loss: 0.2899 - val_loss: 0.2952
Epoch 58/100
1875/1875 ————— 5s 2ms/step - loss: 0.2904 - val_loss: 0.2953
Epoch 59/100
1875/1875 ————— 6s 2ms/step - loss: 0.2910 - val_loss: 0.2952
```

```
<keras.src.callbacks.history.History at 0x794e0d995870>
```

```
# plot the training history
```

```
plt.plot(autoencoder.history.history['loss'])
plt.plot(autoencoder.history.history['val_loss'])
plt.xlabel("epoch")
plt.ylabel("Loss")
```

```
plt.title("Loss on train and test data")
plt.legend(['Train', 'Test'], loc='upper right')
plt.show()
```



```
# Perform prediction on the noisy test data
reconstructed_images = autoencoder.predict(x_test_noisy)

# plot the test image
def validate_image(data):
    random_sample = np.random.choice(data.shape[0], 1, replace=False)
    print(f"Testing on image {random_sample[0]} \n")

    # plot noisy test data
    plt.imshow(x_test_noisy[random_sample].reshape(28, 28))
    plt.title("Noisy Test Image")
    plt.show()

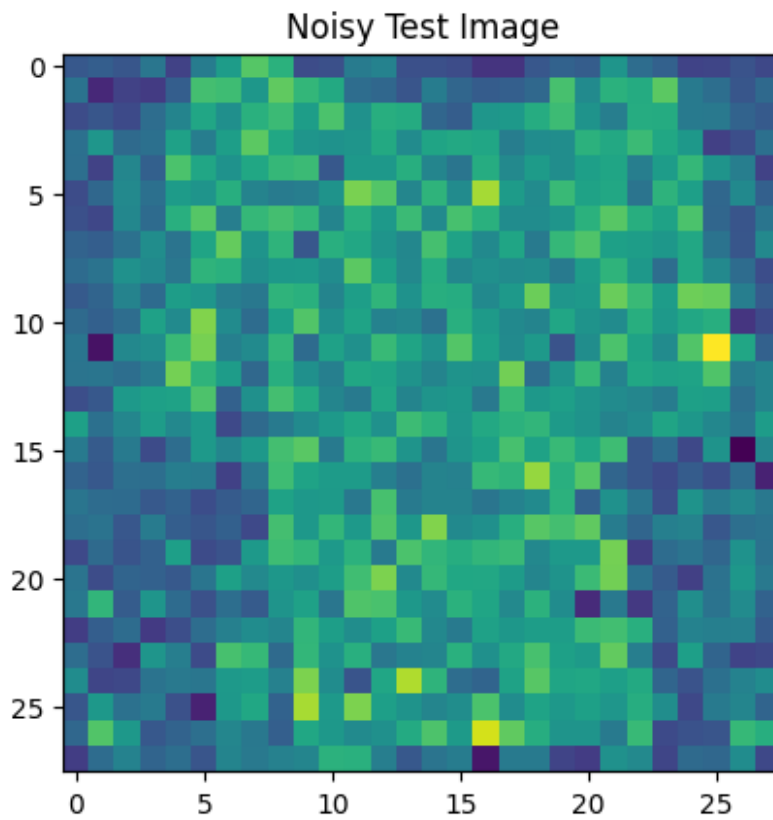
    # plot the reconstructed image (from the predicted data)
    plt.imshow(data[random_sample].reshape(28, 28))
    plt.title("Reconstructed Image")
    plt.show()
```

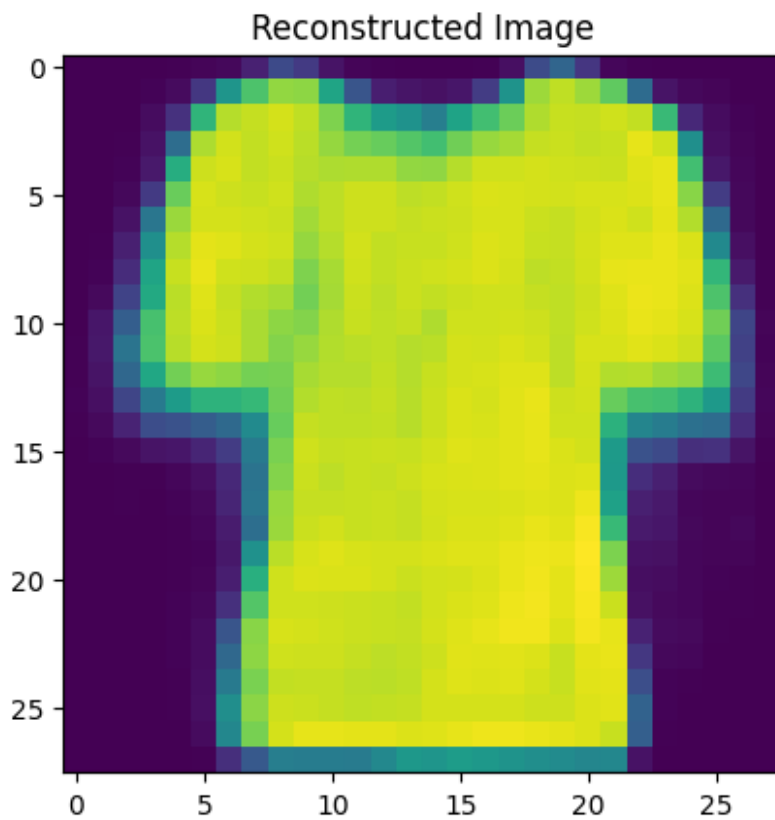
313/313 ————— 0s 1ms/step

```
# Call the validate_image function with the reconstructed images
```

```
validate_image(reconstructed_images)
```

```
Testing on image 9668
```





Video URL:-

https://drive.google.com/file/d/1BqBKmvfV7sYco7PtMAZlhxNBifOJKB4C/view?usp=drive_link